Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп’ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 10

Тема: Файлове введення-виведення та збереження стану за допомогою менеджерів контексту

Мета роботи: Опанувати практичні навички проектування підсистем персистентного збереження даних, файлового введення-виведення та серіалізації структур інформації мовою Python. Навчитися здійснювати потокове читання, фільтрацію та запис текстових файлів через контекстні менеджери with, освоїти структурний парсинг і генерацію звітів у форматах CSV (модуль csv) та JSON (модуль json). Опанувати низькорівневе бінарне позиціонування файлового курсору за допомогою методів seek() і tell(), а також реалізувати валідацію апаратних бінарних файлів за магічними сигнатурами (magic numbers) у задачах комп'ютерної інженерії.

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп’ютерна інженерія»
Освітня програма Комп’ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи та технічна постановка задачі

Мета роботи: опанувати проєктування підсистем персистентного збереження даних і файлового введення-виведення мовою Python: виконувати потокове читання та запис файлів через менеджери контексту оператора with, обробляти табличні дані форматом CSV (модуль csv) і зберігати структурований стан у форматі JSON (модуль json), а також виконувати низькорівневе позиціонування курсору у бінарному файлі методами seek() і tell() із верифікацією заголовка за магічною сигнатурою.

1.1. Індивідуальне завдання (варіант 2)

Предметна область	**Критерій фільтрації CSV та розрахунку**	**Магічна сигнатура BIN**
Моніторинг серверів ЦОД (servers.csv)	Завантаження CPU > 80 %; підрахунок середнього обсягу вільної RAM	0x4E4F4445 (ASCII: «NODE»)

Необхідно реалізувати три завдання: (1) зчитати CSV через csv.DictReader, відібрати рядки за критерієм, обчислити середню величину та зберегти відібрані дані в новий файл через csv.DictWriter; (2) відкрити бінарний файл у режимі ’rb’, перевірити 4-байтну сигнатуру, вилучити блок даних за допомогою seek() і визначити розмір через tell(); (3) зібрати результати в словник стану й зберегти його у файл JSON з відступами indent=4.

1.2. Поля вхідного CSV-файлу servers.csv

Стовпець	**Тип**	**Зміст**
Timestamp	рядок	час зняття показників, формат РРРР-ММ-ДД ГГ:ХХ:СС
Server_ID	рядок	ідентифікатор сервера (SRV-01 … SRV-10)
CPU_Usage_Percent	float	завантаження процесора, %
RAM_Free_GB	float	обсяг вільної оперативної пам’яті, ГБ
Temperature_C	float	температура сервера, °C

Файл містить 10 записів. Серед них навмисно присутні: два граничні значення CPU (80,0 % — не перевищує порогу, 80,1 % — перевищує) для перевірки строгості порівняння, а також один некоректний запис (SRV-05, значення «N/A» замість навантаження CPU) для перевірки стійкості обробки.

1.3. Структура бінарного заголовка firmware_sample.bin

Зміщення	Розмір	Вміст
0 … 3	4 байти	магічна сигнатура 4E 4F 44 45 (ASCII «NODE»), 0x4E4F4445
4	1 байт	Major-версія прошивки: 3
5	1 байт	Minor-версія прошивки: 1
6 … 13	8 байтів	корисний блок даних: 10 20 30 40 50 60 70 80

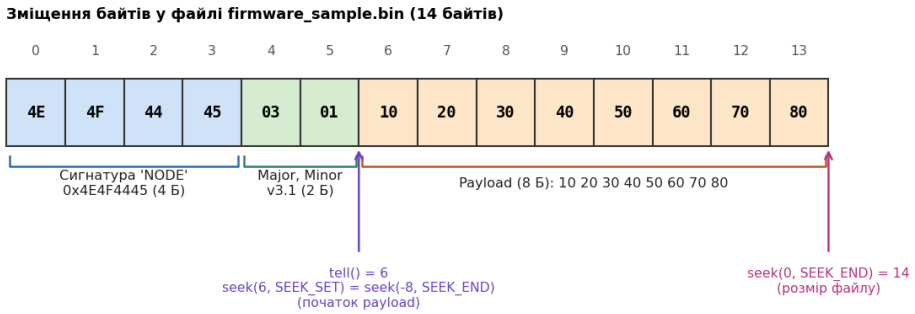


Рисунок 1 — Структура бінарного образу та позиціонування курсору методами seek() і tell()

Нова позиція курсору визначається опорною точкою whence та зміщенням у байтах: whence = 0 (os.SEEK_SET) — початок файлу, 1 (os.SEEK_CUR) — поточна позиція, 2 (os.SEEK_END) — кінець файлу.

$$P_new = P_base(whence) + \Delta_offset$$

Цілісність образу перевіряється порівнянням перших K = 4 байтів файлу з еталонною сигнатурою:

$$Header_actual = file.read(4) \ ?= \ MAGIC_SIGNATURE = 0x4E4F4445$$

2. Алгоритмічний опис файлового конвеєра

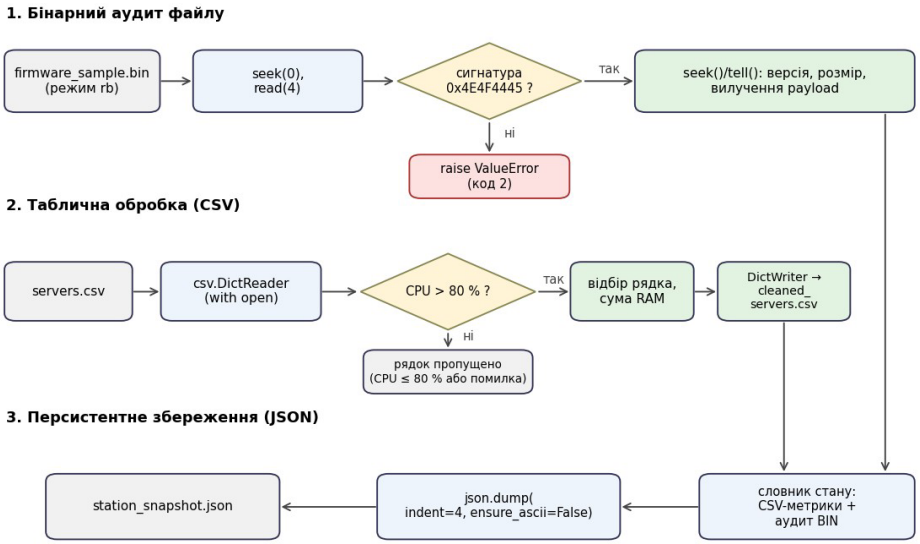


Рисунок 2 — Конвеєр бінарної верифікації, обробки CSV та серіалізації стану у JSON

2.1. Менеджери контексту

Усі файли відкриваються оператором with open(…) as f. Менеджер контексту реалізує протокол __enter__() і __exit__(): при вході захоплюється ресурс, а при виході з блоку (нормальному або через виняток) автоматично скидається буфер методом flush() і закривається файловий дескриптор. Це виключає витік дескрипторів і втрату буферизованих даних, навіть якщо всередині блоку виникне помилка. Кожна функція конвеєра тримає файл відкритим лише на час потрібної операції: CSV читається в одному блоці with, а результат записується в іншому.

2.2. Обробка CSV

Функція process_csv_telemetry виконує такі кроки:

1) перевірити існування вхідного файлу, інакше згенерувати FileNotFoundError;
2) відкрити файл із параметром newline="" (цього вимагає модуль csv для коректної обробки переведень рядка) та створити csv.DictReader: кожен рядок стає словником, де ключами є заголовки стовпців;

3) для кожного рядка перетворити поля CPU_Usage_Percent, RAM_Free_GB, Temperature_C на float; при KeyError, ValueError або TypeError рядок вважається некоректним, його номер і ідентифікатор фіксуються, а обробка триває;
4) якщо CPU > 80,0 (строге порівняння), рядок додається до вибірки разом з обчисленим стовпцем CPU_Excess_Percent = CPU − 80; інакше лічильник нормального навантаження збільшується;
5) записати вибірку у файл cleaned_servers.csv через csv.DictWriter (writeheader() і writerow());
6) обчислити агрегати: середній обсяг вільної RAM та середнє CPU відібраних серверів, сервер із максимальним навантаженням.

Номери рядків відраховуються з 2, оскільки перший рядок файлу містить заголовки. Фільтр порівнює CPU зі строгим «>», тому сервер SRV-03 (80,0 %) не потрапляє до вибірки, а SRV-08 (80,1 %) потрапляє.

2.3. Бінарний аудит

Функція verify_binary_firmware працює в режимі ’rb’, у якому дані повертаються як незмінні послідовності bytes. Алгоритм:

7) зчитати перші 4 байти й порівняти з FIRMWARE_MAGIC_HEADER; при розбіжності згенерувати ValueError із шістнадцятковими значеннями фактичної та очікуваної сигнатур;
8) зчитати наступні 2 байти (Major та Minor); якщо їх менше двох, образ вважається обірваним;
9) викликом tell() зафіксувати позицію кінця заголовка (6);
10) викликом seek(0, os.SEEK_END) перемістити курсор у кінець файлу: метод повертає нову абсолютну позицію, тобто розмір файлу в байтах (14); розмір корисного блоку становить 14 − 6 = 8 байтів;
11) викликом seek(6, os.SEEK_SET) повернутися на початок корисного блоку й прочитати його;

12) для контролю викликом seek(−8, os.SEEK_END) відрахувати ту саму позицію від кінця файлу й порівняти два прочитані блоки: їх збіг (payload_seek_consistent = true) підтверджує коректність обчислення позицій.

2.4. Серіалізація у JSON та обробка помилок

Функція export_system_snapshot об’єднує в одному словнику метадані об’єкта, підсумки CSV-аналізу та результати бінарного аудиту й записує його через json.dump(…, indent=4, ensure_ascii=False). Параметр indent=4 робить файл читабельним, а ensure_ascii=False зберігає кирилицю без екранування (у файл потрапляє, наприклад, опис некоректного рядка). Перетворення Python-типів у JSON однозначне: dict → object, list → array, str → string, int і float → number, bool → true/false.

Головна функція main перехоплює помилки конвеєра й завершує процес відповідним кодом через sys.exit(). Порядок блоків except важливий: FileNotFoundError є підкласом OSError, тому його обробник стоїть першим.

Код	Причина завершення
0	конвеєр виконано успішно
1	FileNotFoundError: не знайдено servers.csv або firmware_sample.bin
2	ValueError: невідома сигнатура образу або обірваний заголовок
3	OSError або csv.Error: інша системна помилка введення-виведення чи розбору CSV

Шляхи до файлів будуються від каталогу самого скрипта: Path(__file__).resolve().parent. Завдяки цьому програма знаходить свої файли незалежно від того, з якого каталогу запущено термінал.

3. Програмний код

3.1. Файл telemetry_pipeline.py

```
"""
Лабораторна робота № 10. Варіант 2.
```

```
Комплексний конвеєр файлового введення-виведення для моніторингу серверів ЦОД:
обробка CSV (фільтр CPU > 80 %, середній обсяг вільної RAM), бінарна верифікація
образу прошивки за магічною сигнатурою 'NODE' (0x4E4F4445) та персистентна
серіалізація підсумкового стану у JSON.
Освітній компонент: Програмування на Python.
"""

import csv
import json
import os
import sys
import time
from pathlib import Path

# Каталог модуля: файли шукаються поруч зі скриптом незалежно від поточного
каталогу терміналу
BASE_DIR = Path(__file__).resolve().parent

# Константа 4-байтної магічної сигнатури бінарного образу (ASCII: 'NODE')
FIRMWARE_MAGIC_HEADER = bytes([0x4E, 0x4F, 0x44, 0x45])
# Розмір заголовка: 4 байти сигнатури + 1 байт Major + 1 байт Minor
FIRMWARE_HEADER_SIZE = 6

# Критерій фільтрації варіанта 2: завантаження процесора вище 80 %
CPU_THRESHOLD_PERCENT = 80.0


def process_csv_telemetry(input_csv_path: Path, output_csv_path: Path) -> dict:
    """
    Зчитує телеметрію серверів через csv.DictReader, відбирає рядки із
завантаженням
    CPU вище порога, рахує середній обсяг вільної RAM відібраних серверів та
записує
    відібрані рядки у новий CSV-файл через csv.DictWriter.
    Некоректні рядки (нечислові значення, відсутні поля) пропускаються й
обліковуються.
    Повертає словник агрегованих метрик.
    """
    if not input_csv_path.exists():
        raise FileNotFoundError(f"Вхідний файл '{input_csv_path.name}' не
знайдено.")

    total_rows = 0
    normal_rows = 0
    malformed_rows: list[str] = []
    selected: list[dict] = []

    # Читання вхідного CSV-файлу через менеджер контексту (newline="" вимагається
модулем csv)
    with open(input_csv_path, mode="r", encoding="utf-8", newline="") as in_file:
        reader = csv.DictReader(in_file)
```

```
        # Нумерація з 2: перший рядок файлу містить заголовки стовпців
        for line_no, row in enumerate(reader, start=2):
            total_rows += 1
            try:
                cpu = float(row["CPU_Usage_Percent"])
                ram_free = float(row["RAM_Free_GB"])
                temperature = float(row["Temperature_C"])
            except (KeyError, ValueError, TypeError):
                # Некоректний рядок не зупиняє конвеєр: фіксуємо номер рядка та
сервер
                malformed_rows.append(f"рядок {line_no} ({row.get('Server_ID',
'?')})")
                continue

            if cpu > CPU_THRESHOLD_PERCENT:
                selected.append({
                    "Timestamp": row["Timestamp"],
                    "Server_ID": row["Server_ID"],
                    "CPU_Usage_Percent": cpu,
                    "RAM_Free_GB": ram_free,
                    "Temperature_C": temperature,
                    "CPU_Excess_Percent": cpu - CPU_THRESHOLD_PERCENT,
                })
            else:
                normal_rows += 1

    # Запис відібраних даних у новий CSV-файл
    fieldnames = ["Timestamp", "Server_ID", "CPU_Usage_Percent", "RAM_Free_GB",
                  "Temperature_C", "CPU_Excess_Percent"]
    with open(output_csv_path, mode="w", encoding="utf-8", newline="") as
out_file:
        writer = csv.DictWriter(out_file, fieldnames=fieldnames)
        writer.writeheader()
        for rec in selected:
            writer.writerow({key: (f"{val:.2f}" if isinstance(val, float) else
val)
                             for key, val in rec.items()})

    count = len(selected)
    stats = {
        "total_read_rows": total_rows,
        "malformed_rows_skipped": len(malformed_rows),
        "normal_load_rows": normal_rows,
        "overloaded_rows_saved": count,
        "cpu_threshold_percent": CPU_THRESHOLD_PERCENT,
        "average_free_ram_gb": round(sum(r["RAM_Free_GB"] for r in selected) /
count, 4) if count else 0.0,
        "average_cpu_percent": round(sum(r["CPU_Usage_Percent"] for r in
selected) / count, 4) if count else 0.0,
        "overloaded_servers": [r["Server_ID"] for r in selected],
```

```
    }
    if selected:
        hottest = max(selected, key=lambda r: r["CPU_Usage_Percent"])
        stats["max_cpu_server"] = {"server_id": hottest["Server_ID"],
                                   "cpu_usage_percent":
hottest["CPU_Usage_Percent"]}
    stats["malformed_details"] = malformed_rows
    return stats


def verify_binary_firmware(bin_path: Path) -> dict:
    """
    Низькорівневий аналіз бінарного образу: верифікація магічної сигнатури,
    вилучення версії, визначення розміру образу через seek()/tell() та вилучення
    блоку корисних даних двома способами (від початку й від кінця файлу).
    """
    if not bin_path.exists():
        raise FileNotFoundError(f"Файл бінарного образу '{bin_path.name}' не
знайдено.")

    with open(bin_path, mode="rb") as b_file:
        # 1. Зчитування перших 4 байтів заголовка та порівняння з еталонною
сигнатурою
        header = b_file.read(4)
        if header != FIRMWARE_MAGIC_HEADER:
            raise ValueError(
                f"Невідомий формат образу: сигнатура {header.hex().upper() or
'(порожньо)'} "
                f"!= {FIRMWARE_MAGIC_HEADER.hex().upper()}"
            )

        # 2. Зчитування версії (байти 4 та 5); обірваний файл вважається
пошкодженим
        version_bytes = b_file.read(2)
        if len(version_bytes) != 2:
            raise ValueError("Образ обірваний: заголовок містить менше 6
байтів.")
        major_ver, minor_ver = version_bytes[0], version_bytes[1]

        # 3. tell() повертає поточну позицію курсору: кінець заголовка (6-й байт)
        header_end = b_file.tell()

        # 4. seek(0, SEEK_END) переміщує курсор у кінець файлу й повертає розмір
у байтах
        file_size_bytes = b_file.seek(0, os.SEEK_END)
        payload_size = file_size_bytes - header_end

        # 5. Вилучення корисного блоку: абсолютне позиціонування від початку
файлу
        b_file.seek(header_end, os.SEEK_SET)
        payload = b_file.read(payload_size)
```

```
        # 6. Контрольне вилучення: відносне позиціонування від кінця файлу
        b_file.seek(-payload_size, os.SEEK_END)
        payload_from_end = b_file.read()

    return {
        "magic_signature_hex": f"0x{header.hex().upper()}",
        "magic_signature_ascii": header.decode("ascii"),
        "firmware_version": f"v{major_ver}.{minor_ver}",
        "total_image_size_bytes": file_size_bytes,
        "header_size_bytes": header_end,
        "payload_size_bytes": payload_size,
        "payload_hex": " ".join(f"{byte:02X}" for byte in payload),
        "is_signature_valid": True,
        "payload_seek_consistent": payload == payload_from_end,
    }


def export_system_snapshot(output_json_path: Path, csv_stats: dict, bin_stats:
dict) -> None:
    """Серіалізує підсумковий стан системи у структурований JSON-документ."""
    snapshot_payload = {
        "station_metadata": {
            "facility_name": "Data Center Server Cluster DC-1",
            "snapshot_generated_at": time.strftime("%Y-%m-%d %H:%M:%S"),
            "pipeline_version": "1.0.0",
        },
        "server_telemetry_summary": csv_stats,
        "node_firmware_audit": bin_stats,
    }

    with open(output_json_path, mode="w", encoding="utf-8") as j_file:
        json.dump(snapshot_payload, j_file, indent=4, ensure_ascii=False)


def main() -> None:
    """Головна точка входу лабораторної роботи."""
    print("=" * 80)
    print(f"{'КОНВЕЄР ФАЙЛОВОГО ВВЕДЕННЯ-ВИВЕДЕННЯ ТА ЗБЕРЕЖЕННЯ СТАНУ
(ЦОД)':^80}")
    print("=" * 80)

    input_csv = BASE_DIR / "servers.csv"
    cleaned_csv = BASE_DIR / "cleaned_servers.csv"
    input_bin = BASE_DIR / "firmware_sample.bin"
    output_json = BASE_DIR / "station_snapshot.json"

    t_start = time.perf_counter()

    try:
        # 1. Обробка CSV телеметрії
```

```
        csv_metrics = process_csv_telemetry(input_csv, cleaned_csv)
        print("Обробка CSV успішна:")
        print(f"  {'Зчитано рядків:':<30}{csv_metrics['total_read_rows']:>5d}")
        print(f"  {'Некоректних (пропущено):':<30}
{csv_metrics['malformed_rows_skipped']:>5d}")
        print(f"  {'Серверів із CPU <= 80%:':<30}
{csv_metrics['normal_load_rows']:>5d}")
        print(f"  {'Серверів із CPU > 80%:':<30}
{csv_metrics['overloaded_rows_saved']:>5d}")
        print(f"  {'Середня вільна RAM:':<30}
{csv_metrics['average_free_ram_gb']:>5.2f} ГБ")
        print(f"  {'Файл збережено:':<30}{cleaned_csv.name}")
        print("-" * 80)

        # 2. Бінарний аудит прошивки
        bin_metrics = verify_binary_firmware(input_bin)
        print("Аудит бінарного образу успішний:")
        print(f"  Магічна сигнатура:     {bin_metrics['magic_signature_hex']}
('{bin_metrics['magic_signature_ascii']}')")
        print(f"  Версія прошивки:       {bin_metrics['firmware_version']}")
        print(f"  Загальний розмір:      {bin_metrics['total_image_size_bytes']}
байтів")
        print(f"  Розмір корисних даних: {bin_metrics['payload_size_bytes']}
байтів")
        print(f"  Корисний блок (HEX):   {bin_metrics['payload_hex']}")
        print(f"  Узгодженість seek():
{bin_metrics['payload_seek_consistent']}")
        print("-" * 80)

        # 3. Серіалізація підсумкового стану у JSON
        export_system_snapshot(output_json, csv_metrics, bin_metrics)
        print("Підсумковий знімок стану збережено у JSON:")
        print(f"  Цільовий файл:         {output_json.name}")
        print("=" * 80)

    except FileNotFoundError as err:        # має стояти перед OSError (є його
підкласом)
        print(f"Помилка файлу: {err}")
        sys.exit(1)
    except ValueError as err:
        print(f"Помилка формату даних: {err}")
        sys.exit(2)
    except (OSError, csv.Error) as err:
        print(f"Системна помилка введення-виведення: {err}")
        sys.exit(3)

    t_elapsed = time.perf_counter() - t_start
    print(f"Час виконання повного конвеєра: {t_elapsed * 1000:.3f} мс\n")


if __name__ == "__main__":
```

```
    main()
```

4. Результати тестування

Запуск виконувався в середовищі Conda ce_lab_env командою python telemetry_pipeline.py.

4.1. Вхідні дані

```
Timestamp,Server_ID,CPU_Usage_Percent,RAM_Free_GB,Temperature_C
2026-08-30 15:00:00,SRV-01,45.2,28.4,52.0
2026-08-30 15:00:00,SRV-02,92.7,6.1,71.5
2026-08-30 15:00:00,SRV-03,80.0,12.0,66.0
2026-08-30 15:00:00,SRV-04,85.3,9.8,69.2
2026-08-30 15:00:00,SRV-05,N/A,16.0,55.0
2026-08-30 15:00:00,SRV-06,97.9,3.2,78.4
2026-08-30 15:00:00,SRV-07,60.4,20.5,58.1
2026-08-30 15:00:00,SRV-08,80.1,10.5,68.0
2026-08-30 15:00:00,SRV-09,88.0,7.4,72.3
2026-08-30 15:00:00,SRV-10,33.8,30.2,49.7
```

Рисунок 3 — Вхідний файл servers.csv

```
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab10> python -c "print(open('firmware_sample.bin','rb')
.read().hex(' ').upper())"
4E 4F 44 45 03 01 10 20 30 40 50 60 70 80
```

Рисунок 4 — Вміст бінарного файлу firmware_sample.bin

4.2. Результат роботи програми

```
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab10> (C:\Users\Janis\miniconda3\S
cripts\activate) ; (conda activate ce_lab_env)
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab10> & 'C:\Users\Janis\miniconda3
\envs\ce_lab_env\python.exe' '"c:/Users/Janis/Desktop/ypok/Програмування на Python. Базовий рівень/lab10/t
elemetry_pipeline.py"'
================================================================================
        КОНВЕЄР ФАЙЛОВОГО ВВЕДЕННЯ-ВИВЕДЕННЯ ТА ЗБЕРЕЖЕННЯ СТАНУ (ЦОД)
================================================================================
Обробка CSV успішна:
  Зчитано рядків:                10
  Некоректних (пропущено):        1
  Серверів із CPU <= 80%:         4
  Серверів із CPU > 80%:          5
  Середня вільна RAM:          7.40 ГБ
  Файл збережено:              cleaned_servers.csv
--------------------------------------------------------------------------------
Аудит бінарного образу успішний:
  Магічна сигнатура:     0x4E4F4445 ('NODE')
  Версія прошивки:       v3.1
  Загальний розмір:      14 байтів
  Розмір корисних даних: 8 байтів
  Корисний блок (HEX):   10 20 30 40 50 60 70 80
  Узгодженість seek():   True
--------------------------------------------------------------------------------
Підсумковий знімок стану збережено у JSON:
  Цільовий файл:         station_snapshot.json
================================================================================
Час виконання повного конвеєра: 30.221 мс
```

Рисунок 5 — Консольний вивід файлового конвеєра

4.3. Згенерований файл cleaned_servers.csv

```
Timestamp,Server_ID,CPU_Usage_Percent,RAM_Free_GB,Temperature_C,CPU_Excess_Percent
2026-08-30 15:00:00,SRV-02,92.70,6.10,71.50,12.70
2026-08-30 15:00:00,SRV-04,85.30,9.80,69.20,5.30
2026-08-30 15:00:00,SRV-06,97.90,3.20,78.40,17.90
2026-08-30 15:00:00,SRV-08,80.10,10.50,68.00,0.10
2026-08-30 15:00:00,SRV-09,88.00,7.40,72.30,8.00
```

Рисунок 6 — Відібрані сервери з навантаженням CPU понад 80 %

4.4. Згенерований файл station_snapshot.json


```
{
    "station_metadata": {
        "facility_name": "Data Center Server Cluster DC-1",
        "snapshot_generated_at": "2026-11-11 06:49:21",
        "pipeline_version": "1.0.0"
    },
    "server_telemetry_summary": {
        "total_read_rows": 10,
        "malformed_rows_skipped": 1,
        "normal_load_rows": 4,
        "overloaded_rows_saved": 5,
        "cpu_threshold_percent": 80.0,
        "average_free_ram_gb": 7.4,
        "average_cpu_percent": 88.8,
        "overloaded_servers": [
            "SRV-02",
            "SRV-04",
            "SRV-06",
            "SRV-08",
            "SRV-09"
        ],
        "max_cpu_server": {
            "server_id": "SRV-06",
            "cpu_usage_percent": 97.9
        },
        "malformed_details": [
            "рядок 6 (SRV-05)"
        ]
    },
    "node_firmware_audit": {
        "magic_signature_hex": "0x4E4F4445",
        "magic_signature_ascii": "NODE",
        "firmware_version": "v3.1",
        "total_image_size_bytes": 14,
        "header_size_bytes": 6,
        "payload_size_bytes": 8,
        "payload_hex": "10 20 30 40 50 60 70 80",
        "is_signature_valid": true,
        "payload_seek_consistent": true
    }
}
```


Рисунок 7 — Повний вміст підсумкового файлу station_snapshot.json

4.5. Перевірка обробки помилок

```
### Тест A: відсутній бінарний файл
Помилка файлу: Файл бінарного образу 'firmware_sample.bin' не знайдено.
[код завершення: 1]


### Тест B: пошкоджена магічна сигнатура (перший байт 0x4E замінено на 0x00)
Помилка формату даних: Невідомий формат образу: сигнатура 004F4445 != 4E4F4445
[код завершення: 2]


### Тест C: обірваний образ (залишилось лише 5 байтів)
Помилка формату даних: Образ обірваний: заголовок містить менше 6 байтів.
```

```
[код завершення: 2]


### Тест D: відсутній вхідний CSV-файл
Помилка файлу: Вхідний файл 'servers.csv' не знайдено.
[код завершення: 1]
```

Рисунок 8 — Реакція конвеєра на відсутні та пошкоджені файли

4.6. Аналіз результатів

Фільтрація CSV. З 10 прочитаних рядків 5 відібрано як перевантажені (SRV-02, SRV-04, SRV-06, SRV-08, SRV-09), 4 мають нормальне навантаження (SRV-01, SRV-03, SRV-07, SRV-10), а 1 некоректний (SRV-05, рядок 6 файлу) пропущено без зупинки конвеєра: 5 + 4 + 1 = 10. Граничні значення оброблено правильно: SRV-03 із завантаженням 80,0 % не перевищує порогу й не потрапив до вибірки, а SRV-08 із 80,1 % потрапив зі значенням CPU_Excess_Percent = 0,10.

Перевірка розрахунків вручну. Середній обсяг вільної RAM відібраних серверів: (6,1 + 9,8 + 3,2 + 10,5 + 7,4) / 5 = 37,0 / 5 = 7,40 ГБ. Середнє завантаження CPU: (92,7 + 85,3 + 97,9 + 80,1 + 88,0) / 5 = 444,0 / 5 = 88,8 %. Найбільше навантаження має SRV-06 (97,9 %), він же має найменший обсяг вільної RAM (3,2 ГБ). Значення збігаються з полями average_free_ram_gb, average_cpu_percent та max_cpu_server у JSON.

Бінарний аудит. Сигнатура 4E 4F 44 45 збігається з очікуваною («NODE»), версія прошивки v3.1. Розмір файлу, визначений викликом seek(0, os.SEEK_END), дорівнює 14 байтам, що відповідає 4 + 2 + 8; позиція кінця заголовка за tell() дорівнює 6, а розмір корисного блоку 14 − 6 = 8 байтів. Блок, прочитаний від початку файлу, збігається з блоком, прочитаним через seek(−8, os.SEEK_END), тому payload_seek_consistent = true.

Обробка помилок. Відсутність файлу дає код 1 (тести A та D), пошкоджена сигнатура 00 4F 44 45 та обірваний образ із 5 байтів дають код 2 (тести B та C). Зазначимо, що в тестах A–C етап CSV уже виконано успішно до виникнення помилки, оскільки обробка CSV виконується першою.

5. Посилання на віддалений репозиторій

Профіль: https://github.com/kefir4ikk

Репозиторій лабораторної роботи: https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab10

6. Висновки

У ході лабораторної роботи реалізовано конвеєр файлового введення-виведення для моніторингу серверів ЦОД. Використання менеджерів контексту with гарантує закриття файлових дескрипторів і скидання буферів навіть при виникненні винятків усередині блоку, що усуває витік ресурсів ядра й втрату даних. Кожна функція конвеєра відкриває файл на мінімально потрібний час і не залишає відкритих дескрипторів.

Робота з CSV через DictReader і DictWriter дозволила звертатися до полів за назвами стовпців, а не за індексами, що робить код стійким до зміни порядку стовпців. Окрема обробка некоректних рядків (SRV-05) показала, що конвеєр не повинен падати через один дефектний запис, але має його облікувати. Формат JSON з параметрами indent=4 та ensure_ascii=False забезпечив читабельне, структуроване й переносне збереження підсумкового стану, яке може бути прочитане будь-якою системою.

Робота з бінарним файлом показала, що верифікація магічної сигнатури є дешевою перевіркою, яка відсікає файли чужого формату та пошкоджені образи ще до їхньої обробки; перевірка довжини заголовка додатково відсікає обірвані файли. Методи seek() і tell() дозволили визначити розмір файлу й вилучити корисний блок без читання всього файлу, а збіг блоків, отриманих абсолютним і відносним від кінця позиціонуванням, підтвердив правильність обчислення зміщень. Обробка помилок через коди завершення (1, 2, 3) робить утиліту придатною для використання в скриптах автоматизації.

Контрольні питання

1. Як влаштований протокол менеджера контексту (__enter__ та __exit__) у мові Python? Чому використання інструкції with open(...) гарантує закриття файлу навіть у разі збудження винятку ZeroDivisionError під час запису?

Під час виконання with expr as name інтерпретатор обчислює вираз, викликає метод __enter__() отриманого об’єкта й прив’язує його результат до імені (для файлу це сам файловий об’єкт). Після виходу з блоку — нормального чи через виняток — завжди викликається __exit__(exc_type, exc_val, exc_tb). Для файлів він виконує flush() і close(), звільняючи дескриптор. Якщо в блоці виникає ZeroDivisionError, інтерпретатор спочатку викликає __exit__ (файл закривається, буфер скидається), і лише потім виняток продовжує поширюватись угору, оскільки __exit__ файлу повертає хибне значення й не придушує виняток. Це еквівалентно конструкції try ... finally: f.close(), але лаконічніше та безпечніше. Власні менеджери можна створювати класом із методами __enter__ та __exit__ або декоратором contextlib.contextmanager.

2. У чому полягає різниця між текстовим ('r', 'w') та бінарним ('rb', 'wb') режимами відкриття файлів? Чому в текстовому режимі використання довільних зміщень у методі file.seek(offset) є забороненим для багатобайтових кодувань UTF-8?

У текстовому режимі потік автоматично декодує байти в рядки str за вказаним кодуванням (за замовчуванням залежним від платформи, тому слід явно задавати encoding="utf-8") і перетворює символи кінця рядка (універсальні переведення рядка). У бінарному режимі дані повертаються й записуються як bytes без жодних перетворень. UTF-8 є кодуванням зі змінною довжиною символа (1–4 байти). Довільне байтове зміщення може потрапити всередину багатобайтового символа, і декодування дасть помилку або хибний результат. Тому в текстовому режимі seek() приймає лише 0 або значення,

раніше повернуте методом tell() (це непрозорий маркер, що враховує стан декодера); відносне зміщення від поточної позиції чи від кінця з ненульовим значенням спричиняє io.UnsupportedOperation. Для довільного позиціонування файл відкривають у бінарному режимі.

3. Поясніть призначення та інженерне використання магічних сигнатур (magic numbers) у бінарних файлах. Що повертає метод file.seek(0, os.SEEK_END)?

Магічна сигнатура — це фіксована послідовність байтів на початку файлу, що однозначно ідентифікує його формат (наприклад, %PDF у PDF, байти 7F 45 4C 46 у ELF, PK у ZIP, 89 50 4E 47 у PNG, а в цій роботі NODE для образу вузла). Вона дозволяє швидко перевірити тип і цілісність вмісту незалежно від розширення імені файлу, не розбираючи його повністю. У інженерних задачах це захищає від завантаження в пристрій чужого або пошкодженого образу прошивки. Виклик file.seek(0, os.SEEK_END) переміщує курсор у кінець файлу й повертає нову абсолютну позицію, тобто розмір файлу в байтах; саме так у лабораторній роботі визначено розмір 14 байтів.

4. Чим відрізняються методи модуля json: json.dump() від json.dumps() та json.load() від json.loads()? Які типи даних Python не підтримують пряму серіалізацію у формат JSON без власного енкодера?

Функція json.dump(obj, fp) записує серіалізований документ у файловий об’єкт fp, а json.dumps(obj) повертає його як рядок str. Функція json.load(fp) читає й розбирає JSON із файлового об’єкта, а json.loads(s) розбирає рядок (або bytes). Суфікс s означає string. Пряма серіалізація підтримується для dict, list, tuple (записується як масив і після читання повертається як list), str, int, float, bool та None. Не серіалізуються без параметра default= або власного JSONEncoder: set і frozenset, bytes, datetime, Decimal, complex, Path та екземпляри користувацьких класів. Ключі словника мають бути рядками (числові ключі перетворюються на рядки, а кортежі спричиняють помилку).

5. Опишіть переваги використання класу csv.DictReader над базовим csv.reader при роботі з табличними даними, що мають заголовки стовпців.

DictReader повертає кожен рядок як словник, ключами якого є заголовки стовпців, тому доступ виконується за іменем (row["CPU_Usage_Percent"]), а не за числовим індексом. Код стає самодокументованим і стійким до зміни порядку чи додавання стовпців, а також простіше передавати рядок у DictWriter. Додаткові можливості: атрибут fieldnames, параметри restkey та restval для рядків з надлишком чи нестачею полів (за нестачі відсутні значення дорівнюють None, що в роботі спричиняє TypeError і трактується як некоректний рядок). Недоліки: невеликі накладні витрати на створення словника для кожного рядка, а всі значення залишаються рядками, тому їх потрібно явно перетворювати на числа.

6. Що таке системна буферизація введення-виведення, і як параметр buffering функції open() впливає на продуктивність запису великих масивів телеметрії?

Операції запису спочатку накопичуються у буфері оперативної пам’яті й надсилаються в ОС порціями, щоб зменшити кількість дорогих системних викликів. Розмір буфера за замовчуванням дорівнює io.DEFAULT_BUFFER_SIZE (зазвичай 8192 байти) або розміру блоку пристрою. Параметр buffering=-1 вмикає це значення за замовчуванням, buffering=0 вимикає буферизацію (лише для бінарного режиму), buffering=1 увімкає рядкову буферизацію (лише для текстового режиму), значення більше 1 задає розмір буфера в байтах. Більший буфер зменшує кількість системних викликів і підвищує пропускну здатність при записі великих масивів телеметрії; проте при аварійному завершенні програми дані, що не скинуті, можуть бути втрачені. Тому для надійності використовують flush() та os.fsync(), а оператор with скидає буфер автоматично при виході з блоку.